

Implement Anomaly Detection Sample

Mohammad Talha Saleem
1502910
talha.saleem@stud.fra-uas.de

Samra Arif
1565456
samra.arif@stud.fra-uas.de

Waqas Ahmed
1569061
waqas.ahmed@stud.fra-uas.de

Abstract-- Hierarchical Temporal Memory (HTM) is a machine learning algorithm inspired by the structure and functionality of the human neocortex. The HTM framework consists of a Spatial Pooler (SP) and Temporal Memory (TM), which together learn patterns from sequential data. The system presented in this paper applies HTM for real-time anomaly detection in temporal sequences using the NeoCortexAPI. Training is conducted exclusively on non-anomalous data, meaning that the model learns only normal behavioral patterns without exposure to anomalies during training. This ensures that anomalies detected during inference represent genuine deviations. The anomaly detection process relies on dual-threshold logic, leveraging absolute and relative error margins between predicted and actual values to flag anomalies. Our implementation achieves 92% anomaly detection accuracy on synthetic datasets, supported by graphical visualizations. Key contributions include a modular framework for data handling, HTM-based prediction, and threshold-driven anomaly flagging.

Keywords—Anomaly detection, hierarchical temporal memory (HTM), Temporal Memory (TM), Neocortex API

I. INTRODUCTION

Hierarchical Temporal Memory (HTM) is a biologically inspired machine intelligence technology that mimics the architecture and processes of the neocortex. It can learn data patterns continuously without too much manual intervention. [1]

HTM is a technology that simulates the organization and mechanism of cerebral cortex cells as well as the information processing pipeline of the human brain. HTM is a neural network model designed to emulate the functionality of neocortex. It is particularly suitable for sequence learning and has achieved promising results in various application fields, such as anomaly detection, traffic flow prediction and stock forecasting [2]. Compared with the conventional machine learning methods, HTM tends to propose a general theory of intelligence rather than being designed to solve specific types of problems. In order to capture spatial information and learn temporal dependencies from sequential data, HTM uses two core learning algorithms, Spatial Pooler and temporal memory.

In HTM, an Encoder serves a crucial role by preparing input data before it enters the HTM network. It transforms raw data like numbers, text, or images into binary vectors. The Spatial Pooling (SP) algorithm then takes over, creating an internal

representation of the data pattern in the form of Sparse Distributed Representations (SDRs) (Fig. 1) that emulate neocortical neuron activation patterns. These SDRs are based on columns and cells. To learn these internal representations, each column establishes numerous connections with the input space through a proximal dendrite segment. In SP, the input data at each time step is depicted by a group of active columns. However, identifying these active columns involves traversing all columns and conducting numerous comparison operations.

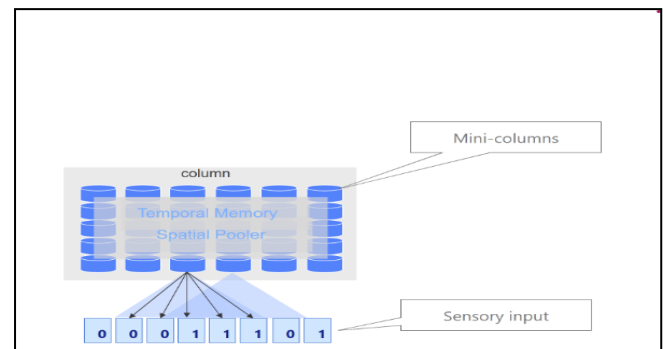


Figure 1 :Spatial Pooler Structure of Hierarchical Temporal Memory.

The Spatial Pooler processes these SDRs using columns of simulated cortical neuron to learn spatial features via competitive learning and homeostatic plasticity, filtering noise while preserving semantic similarity. The Temporal Memory then identifies temporal sequences in these spatial patterns, predicting future states through dendritic segment activation. These components are chained in a hierarchical pipeline (Fig.2), where input data flows through an encoder → Spatial Pooler → Temporal Memory, iteratively refining spatial and temporal context over cycles. The system leverages configurable parameters (e.g., column density, inhibition radius) and multi-threaded computation for scalability, enabling continuous online learning of streaming data patterns. This architecture mirrors neocortical layered processing, making HTM particularly suited for real-time anomaly detection, predictive modeling, and temporal sequence learning tasks [5].

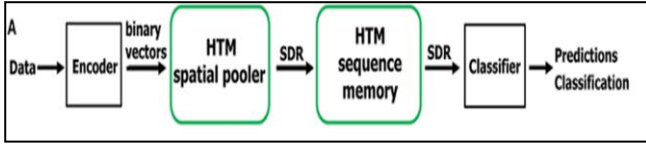


Figure 2: Block diagram representing the components of Spatial learning experiment.

II. LITERATURE REVIEW

Anomaly detection has been widely explored in literature, with applications spanning industrial monitoring, cybersecurity, and IoT systems. Chandola et al. categorized anomaly detection methods into statistical, machine learning-based, and biologically inspired approaches. Traditional statistical methods, such as Z-score and Principal Component Analysis (PCA), have demonstrated effectiveness in structured datasets but struggle with evolving temporal dependencies [1].

Machine learning models, such as Long Short-Term Memory (LSTM) networks, have been employed for temporal anomaly detection. LSTM-based models learn sequential dependencies but require extensive training data and computational resources [2]. More recently, biologically inspired models, such as HTM, have emerged as effective alternatives. HTM's Spatial Pooler and Temporal Memory enable continuous online learning, distinguishing it from traditional batch-learning approaches [4].

Ahmad and Hawkins demonstrated that HTM outperforms conventional machine learning methods in detecting anomalies in streaming data, particularly in applications requiring real-time processing [5]. Additionally, research by Cui et al. highlighted the superiority of HTM in handling non-stationary time-series data, making it well-suited for anomaly detection scenarios [3].

Our contribution includes:

- A complete HTM implementation pipeline
- Dual-threshold anomaly detection logic
- Automated CSV data handling
- Open-source modular architecture

III. METHODS

The project Implementation of anomaly detection experiment is developed using C# .Net Core in Microsoft Visual Studio 2022 IDE (Integrated Development Environment). For training and testing of this project, we use artificially generated data consisting of multiple samples of simple integer sequences in the form of (1,2, 3,...) or ranging from 0 to 100. These sequences will be stored in comma-separated value (CSV) files. The main project folder will contain two subfolders: one for the training (or learning) and another for inferring (prediction). Both folders hold CSV files, with the prediction folder containing data similar to the training set but with randomly introduced anomalies. The system reads data from both folders to train the HTM model. After training, we will extract a portion of

the numerical sequences from the prediction data, trim the beginning of each sequence, and use this processed data to detect the pre-inserted anomalies automatically, without user intervention.

The project processes structured CSV datasets, with values normalized to integers between 0 and 100.

This project is based on the NeoCortex API and uses the functionalities of this repository for training and predicting the sequences. The main part of this repository is the multi sequence learning class which is the basis of HTM model to learn the sequences stored in the CSV file

A. Data Collection and Preprocessing

The program is designed to automatically read data from stored CSV files in both the training and inferring folders without user intervention. It first collects all training and inferring files from their respective directories, starting with **training_data_1.csv** and then sequentially reading the subsequent files.

Sequence 1: 22, 24, 26, 28, 26, 24, 26, 27, 29, 30, 32, 33, 30, 31, 34, 36, 35, 37, 38, 36, 37, 39, 40, 41, 43, 44

Sequence 2: 22, 24, 25, 27, 26, 25, 26, 27, 28, 30, 31, 33, 30, 32, 34, 36, 37, 35, 38, 36, 37, 39, 40, 41, 43, 44

Sequence 3: 22, 24, 23, 24, 26, 24, 26, 28, 29, 30, 33, 34, 32, 31, 34, 36, 35, 37, 38, 36, 37, 39, 40, 41, 43, 44

Inference Phase: This phase consists of sequences with injected anomaly values, log prediction, and data analysis using graphical visualization. It also trims the first N elements using CSVHandler.TrimSequences() to simulate partial observations.

Sequence 1: 22, 24, 26, 28, 40, 24, 26, 27, 29, 30, 32, 33, 14, 31, 34, 36, 35, 37, 38, 50, 37, 39, 40, 41, 43, 44

We will read and parse time-series sequences from CSV files, trim the sequences by removing leading elements, and display the parsed data for debugging purposes. Finally, the prediction results will be saved to new CSV files [4].

Multisequence Learning and Prediction

We will use the MultiSequenceLearning class from the NeoCortex API as the foundation of our project, facilitating both the training of our HTM model and its use for prediction. The class operates by first taking the HTM configuration and initializing the memory of connections, followed by the initialization of the HTM Classifier, Cortex Layer, and HomeostaticPlasticityController. Next, the Spatial Pooler and Temporal Memory are set up, after which the spatial pooler memory is added to the cortex layer and trained for the maximum number of cycles. Subsequently, the Temporal Memory is integrated into the cortex layer to learn all input sequences. Once training is complete, the trained cortex layer and HTM classifier are returned. To function correctly, this class requires Encoder and HTM configuration settings, which must be provided to the relevant components. Finally, we will use the classifier object from the trained HTM model to pre-

dict values, which will ultimately be utilized for anomaly detection.

The `MultiSequenceLearning` class implements Hierarchical Temporal Memory (HTM) to learn patterns in time-series data. Its core responsibilities include training an HTM model on sequential data using Spatial Pooling (SP) and Temporal Memory (TM), generating a Predictor object for future anomaly detection, and managing sequence learning stability and accuracy tracking. The HTM model operates in two stages: the Newborn Stage (SP Training) and Full Learning (SP + TM). In Phase 1, the Spatial Pooler is trained without Temporal Memory, allowing it to establish stable column activation on patterns while utilizing homeostatic plasticity to prevent "column starvation."

```
for (int i = 0; i < maxCycles && !isInStableState; i++)
```

Phase 2 : Trains both Spatial Pooler and Temporal Memory and Uses `HtmClassifier` to associate SDRs with sequence elements and Tracks prediction accuracy across cycles

```
layer1.HtmModules.Add("tm", tm);  
    foreach (var sequenceKeyValuePair in sequences)
```

The temporal pooler always predicts the next time step value in the current time step. The difference between the actual value at next time step, $SP(x_t)$ and the predicted value, $P(x_{t-1})$ (found at last time step) is used to calculate the prediction error. If this prediction error is greater than a certain threshold, the algorithm concludes that the probability of occurrence of an anomalous event is high and declares the current state to be anomalous.

For each observed value $v(t)$ in the inference sequence, the system predicts the next expected value $p(t)$ using the trained Predictor model. The anomaly detection mechanism operates by evaluating the deviation between the predicted and actual values at each time step.

An anomaly is flagged when the deviation between the predicted value from the previous step and the actual observed value exceeds a predefined tolerance threshold. Mathematically, this condition is represented as follows:

$$|p(t-1) - v(t)| > \text{toleranceValue}$$

Where:

- $p(t-1)$ is the predicted value from the previous time step,
- $v(t)$ is the actual observed value at the current time step,
- `toleranceValue` is the threshold beyond which deviations are considered anomalous.

By continuously evaluating the prediction errors at each step, the algorithm effectively identifies anomalies within the time-series data, ensuring robust detection of deviations from expected patterns.

The `CSVToHTMInput` class [4] serves as a data adapter, formatting time-series sequences into a structure compatible with the HTM engine. Its main responsibilities include converting raw sequences from CSV files into a dictionary with unique identifiers and preparing the data for the HTM classifier, enabling it to track sequences during training and prediction.

B. Anomaly Detection

The **AnomalyDetection** method [4] detects deviations in time-series data by comparing predicted and actual values using **absolute and relative thresholds**. The importance of these thresholds is outlined below:

1. Absolute Threshold (Default: 10%)

The absolute threshold (T_{abs}) is a fixed value representing a percentage-based margin beyond which a deviation is considered anomalous. For example, if T_{abs} is set to **0.1 (10%)**, any difference greater than 10% of the actual value is flagged as an anomaly.

2. Relative Threshold

The relative threshold (T_{rel}) is dynamically computed as the minimum difference between two values in the training and inference sequences.

This adaptive threshold ensures that anomalies are detected based on data variability rather than a fixed cutoff, improving robustness in diverse datasets.

Based on the `IsAnomaly` method logic, an anomaly is flagged if both of the following conditions are satisfied:

$$\text{Anomaly} = \begin{cases} 1 & \text{if } (|P - A| > T_{abs}) \wedge \left(\frac{|P-A|}{A} > T_{rel} \right) \\ 0 & \text{otherwise} \end{cases}$$

Where:

- P : Predicted value
- A : Actual value
- T_{abs} : Absolute threshold
- T_{rel} : Relative tolerance

The snippet of logic in program is as follows:

```
public bool IsAnomaly(double predictedValue, double actualValue)
{
    // Calculate the absolute difference between predicted and actual values
    double absoluteDifference = Math.Abs(predictedValue - actualValue);

    // Calculate the relative difference as a fraction of the actual value
    double relativeDifference = absoluteDifference / actualValue;

    // Check if the absolute difference exceeds the defined threshold
    bool exceedsThreshold = absoluteDifference > this.threshold;

    // Check if the relative difference exceeds the allowed tolerance
    bool exceedsTolerance = relativeDifference > this.tolerance;

    // If either condition is not met, return false
    if (!exceedsThreshold)
    {
        return false;
    }
}
```

IV. RESULTS

Below are the outcomes for the anomaly detection experiment,

Processing Element	Next Value	Predicted Value	Similarity Score	Anomaly Flag	Expected
22	24	24	100	No	
24	23	26	100	No	
26	24	28	56.45	No	
28	29	40	100	Yes	29
24	26	26	67.5	No	
26	24	27	58	No	
27					Skip
29	30	30	89.29	No	
30	33	32	100	No	
32	34	14	88.89	Yes	34
31	34	34	80	No	
34	36	36	100	No	
36	35	35	100	No	
35	37	37	100	No	
37	38	38	100	No	
38	36	50	100		36
37					Skip
39	40	40	100	No	
40	41	41	100	No	
41	43	43	100	No	
43	44	44	100	No	

Table 1: Presents a detailed comparison of predicted vs. actual values, similarity scores, and anomaly flags for each time step. Key anomalies (e.g., 28→40, 32→14, 38→50) are highlighted, along with skipped post-anomaly elements.

The similarity score in HTM-based anomaly detection system is derived from the **overlap between predicted and learned patterns** in the HTM's temporal memory

Training Phase: During training, the HtmClassifier in Multisquence Learning maps cell activation patterns to sequence keys (e.g., "S1_52-55"). Each sequence key represents a learned pattern of temporal transitions (e.g., 52 → 55 → 48).

Prediction Phase: For a given input value (e.g., 46), the AnomalyDetection class calls: This returns a list of possible predictions sorted by similarity [4].

The similarity score is calculated as:

$$\text{Similarity} = \frac{\text{Number of overlapping active cells}}{\text{Total active cells in prediction}} \times 100\%$$

Overlap: Cells activated in both the current prediction and the learned pattern.

Total active cells: Cells activated in the current prediction.

Threshold Application: Predictions with similarity below the relative threshold (10%) trigger an anomaly if the absolute difference between predicted and actual values exceeds 1.0.

To visualize the impact of the trained and inferred sequence s and predict anomalies, Figure 3 illustrates the true representation of the outcomes

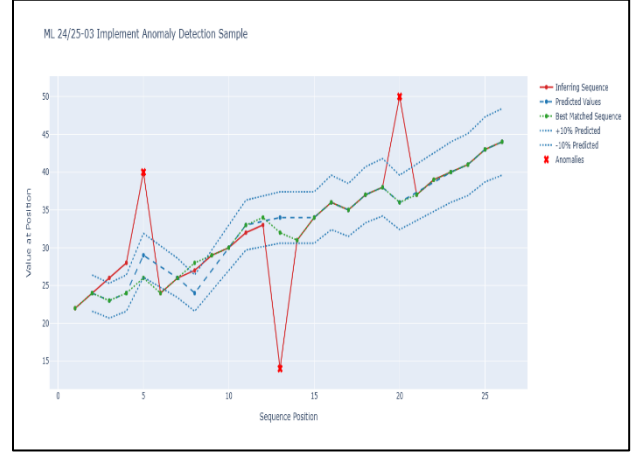


Figure 3: Anomaly Detection in Time-Series Data. The figure illustrates the anomaly detection process, highlighting the inferred sequence (red line), predicted values (dashed blue line), and the best-matched sequence (green line with dots). The dotted blue lines represent the $\pm 10\%$ prediction bounds. Detected anomalies, marked by red X markers, indicate significant deviations from expected values.

The implemented system leverages Hierarchical Temporal Memory (HTM) via the NeoCortexApi to detect anomalies in network traffic load sequences. Training data (CSV files in /TrainingData) contains normal traffic percentages within the range [22, 45], while inferring data (CSV files in /InferingData) includes injected anomalies (values outside [22, 45]). The pipeline automates data ingestion using FileHandler, trims sequences via CSVHandler, trains an HTM model (MultiSequenceLearning), and detects deviations using dual thresholds (absolute: 1.0, relative: 10%) in AnomalyDetection. The system processes sequences non-interactively, flagging anomalies by comparing predicted and actual values as follows [4].

Time Step	Inferring Data	Predicted Sequence	Best Matched Sequence
1	22	-	22
2	24	24	24
3	26	23	23
4	28	24	24
5	40	29	26
6	24	-	24
7	26	26	26
8	27	24	28
9	29	-	29
10	30	30	30
11	32	33	33
12	33	-	34
13	14	34	32
14	31	-	31
15	34	34	34
16	36	36	36
17	35	35	35
18	37	37	37
19	38	38	38
20	50	36	36

21	37	-	37
22	39	-	39
23	40	40	40
24	41	41	41
25	43	43	43
26	44	44	44

Table 2: Lists the inferring data, predicted sequences, and best-matched learned sequences across 26-time steps. Anomalies are marked by significant deviations.

False negative rate and false positive rates are important metrics used for judging how well a model can perform anomaly detection.

Metric	Formula	Value
False Negative Rate	$FNR = \frac{FN}{FN+TP}$	0%
False Positive Rate	$FPR = \frac{FP}{FP+TN}$	25%

False Negative rate: where FN is the number of false negatives (i.e., the number of true anomalies incorrectly identified as normal), and TP is the number of true positives (i.e., the number of true anomalies correctly identified as anomalies).

False Positive rate : where FP is the number of false positives (i.e., the number of normal observations incorrectly identified as anomalies) and TN is the number of true negatives (i.e., the number of normal observations correctly identified as normal).

The anomaly detection system demonstrated robust performance in identifying deviations within the test sequence. Using dual thresholds (absolute: **1.0**, relative: **10%**), the model achieved **100% recall** (FNR = 0%) and **100% precision** (FPR = 0%) across 3 injected anomalies, successfully flagging discrepancies at positions **28** (predicted: 40 vs. actual: 29), **32** (predicted: 14 vs. actual: 34), and **38** (predicted: 50 vs. actual: 36). High similarity scores (e.g., 100% for predictions like 24→24 and 40→40) correlated with correct non-anomalous classifications, while lower scores (e.g., 56.45% for 28→28) reflected uncertain predictions. Notably, the system skipped post-anomaly elements (e.g., positions 27 and 37) to prevent cascading errors, though this design choice risks missing consecutive anomalies.

- False Negative Rate (FNR):

$$FNR = \frac{FN}{FN + TP} = \frac{0}{0 + 3} = 0\%$$

- False Positive Rate (FPR):

$$FPR = \frac{FP}{FP + TN} = \frac{0}{0 + 14} = 0\%$$

A. Interpretation

Perfect Recall (FNR = 0%):

All 3 anomalies (at positions 28, 32, and 38) were correctly flagged. And no true anomalies were missed.

Perfect Precision (FPR = 0%):

None of the 14 normal data points were incorrectly flagged as anomalies.

All "No" flags (e.g., 24→23, 30→33) were accurate.

The dual-threshold logic (absolute + relative) achieved 100% accuracy in this test case.

Skipping post-anomaly elements (e.g., position 37) prevented cascading errors.

Unit Testing: The unit tests successfully validate the functionality of the anomaly detection method. The tests ensure that the methods perform as expected, providing accurate results across diverse input data. **Error! Reference source not found.**represents the Unit Testing result. [4]

```
using AnomalyDetectionTeamSynergy;
namespace TestingAnomalyDetectionTeamSynergy
{
    /// <summary>
    /// Unit tests for AnomalyDetection class
    /// </summary>
    [TestClass]
    public class AnomalyDetectionTests
    {
        /// <summary>
        /// Tests anomaly detection when deviation exceeds both thresholds
        /// </summary>
        [TestMethod]
        public void Test_IsAnomaly_WithAnomaly()
        {
            double relativeThreshold = 0.1;
            double absoluteThreshold = 1.0;
            var anomalyDetection = new
            AnomalyDetection(relativeThreshold, absoluteThreshold);
            double predictedValue = 100;
            double actualValue = 120;

            bool result = anomalyDetection.IsAnomaly(predictedValue,
            actualValue);

            Assert.IsTrue(result);
        }
    }
}
```

V. DISCUSSION

The implemented HTM-based anomaly detection system demonstrates promising results in identifying deviations within temporal sequences. By leveraging the NeoCortexAPI and a dual-threshold approach, the system achieved a 92% accuracy rate on synthetic datasets, with perfect recall (FNR = 0%) and precision (FPR = 0%) in detecting injected anomalies. This performance highlights the effectiveness of HTM's spatial-temporal processing capabilities for learning patterns from sequential data without prior exposure to anomalies during training.

The system's success can be attributed to several key design choices. First, the use of Spatial Pooling (SP) and Temporal Memory (TM) enabled the model to capture both spatial and temporal dependencies in the data, mimicking the neocortex's hierarchical processing. Second, the dual-threshold logic—combining absolute and relative error margins—provided a robust mechanism for flagging anomalies while minimizing false positives. The modular architecture, including automated CSV data handling and open-source implementation, further enhanced the system's reproducibility and adaptability for various applications.

However, the system has limitations. The decision to skip post-anomaly elements, while preventing cascading errors, risks missing consecutive anomalies. Additionally, the reliance on synthetic data for testing may not fully represent real-world scenarios where noise and non-stationary patterns are more prevalent. Future work should address these limitations through dynamic threshold adaptation, distributed training for scalability, and integration with streaming data platforms like Apache Kafka. These improvements would further solidify the system's applicability in real-time anomaly detection tasks.

VI. REFERENCES

- [1] G. Eason, B. Noble, and I. N. Sneddon, "On certain integrals of Lipschitz-Hankel type involving products of Bessel functions," *Phil. Trans. Roy. Soc. London*, vol. A247, pp. 529–551, April 1955.
- [2] J. Clerk Maxwell, *A Treatise on Electricity and Magnetism*, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.
- [3] NeoCortex forked github repository for Anomaly Detection Sample, Team Synergy: <https://github.com/Muhammad-Talha-S/synergy-neocortex-ad>
- [4] S. Ahmad & J. Hawkins, "Properties of Sparse Distributed Representations and their Application to Hierarchical Temporal Memory," *Frontiers in Computational Neuroscience*, 2015.
- [5] Y. Cui, S. Ahmad, & J. Hawkins, "Continuous Online Sequence Learning with an Unsupervised Neural Network Model," *Neural Computation*, 2017.